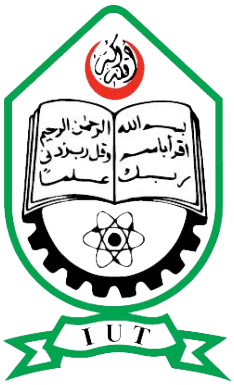

(CSE 4802: Compiler Design Lab)

LAB 4

Report on YACC Parser for Scientific Calculator Expressions

Submitted by:
Md. Mosharrifur Rahman Ratul
Student ID: 210041213
Section: 2A
BSc in CSE, Dept of Computer Science and Engineering
Islamic University of Technology (IUT)

June 13, 2026

Contents

1 Introduction

A parser is a component of a compiler that checks whether a sequence of tokens follows the grammar rules of a language. It receives tokens from the lexical analyzer and builds the syntactic structure of the input.

Flex is a tool used to generate lexical analyzers, while Yacc (Yet Another Compiler Compiler) is used to generate parsers from a given grammar. In this lab, Flex and Yacc are used together to implement a scientific calculator capable of evaluating mathematical expressions and functions.

2 Approach

The lexical analyzer is implemented using Flex to recognize numbers, operators, functions, constants, and identifiers. The parser is implemented using Yacc to define the grammar and evaluation rules of mathematical expressions.

The parser supports arithmetic operations, scientific functions, constants, variable assignments, and error handling. Input expressions are read from a file and the calculated results are written to an output file. I also checked the robustness of the YACC parser by intentionally giving wrong input and the parser can successfully detect them and report the errors properly.

3 Code Implementation

3.1 Lexical Analyzer (`lexer.l`)

Here is the code for lexer.l file.

Listing 1: lexer.l

```
1
2  %{
3  #include "parser.tab.h"
4  #include <stdlib.h>
5  #include <string.h>
6
7  extern YYSTYPE yylval;
8  %}
```

```
9
10  %%
11
12  [0-9]+\.[0-9]+ {
13      yylval.dval = atof(yytext);
14      return DOUBLE;
15  }
16
17  [0-9]+ {
18      yylval.ival = atoi(yytext);
19      return INTEGER;
20  }
21
22  sin    { return SIN; }
23  cos    { return COS; }
24  tan    { return TAN; }
25  sqrt   { return SQRT; }
26  log    { return LOG; }
27
28  [A-Za-z_][A-Za-z0-9_]* {
29      strncpy(yylval.sval, yytext, sizeof(yylval.sval) - 1);
30      yylval.sval[sizeof(yylval.sval) - 1] = '\0';
31      return ID;
32  }
33
34  "+"     { return '+'; }
35  "-"     { return '-'; }
36  "*"     { return '*'; }
37  "/"     { return '/'; }
38  "^"     { return '^'; }
39  "="     { return '='; }
40  "("     { return '('; }
41  ")"     { return ')'; }
42
43  [ \t\r]+   { /* ignore whitespace */ }
44  \n         { return '\n'; }
45
46  .          { return yytext[0]; }
47
48  %%
49
50  int yywrap(void) {
51      return 1;
52  }
```

3.2 Parser (`parser.y`)

Listing 2: parser.y

```
1
2  %{
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <math.h>
6  #include <string.h>
7
8  #define MAX_VAR 1000
9
10 void yyerror(char *s);
11 int yylex();
12
13 extern FILE *yyin;
14
15 /* Output file for results */
16 FILE *output = NULL;
17
18 typedef struct
19 {
20     char name[100];
21     double value;
22 } Symbol;
23
24 Symbol table[MAX_VAR];
25 int symbolCount = 0;
26
27 /* ---------- Symbol Table ---------- */
28
29 void saveSymbols()
30 {
31     FILE *fp = fopen("symbols.txt","w");
32
33     if(!fp)
34         return;
35
36     for(int i=0;i<symbolCount;i++)
37     {
38         fprintf(fp,"%s %lf\n",
39                 table[i].name,
40                 table[i].value);
41     }
42
43     fclose(fp);
44 }
45
46 void loadSymbols()
47 {
48     FILE *fp = fopen("symbols.txt","r");
49
```

```
50      if(!fp)
51          return;
52
53      while(fscanf(fp,"%s %lf",
54                   table[symbolCount].name,
55                   &table[symbolCount].value)==2)
56      {
57          symbolCount++;
58      }
59
60      fclose(fp);
61  }
62
63  double getSymbol(char *name)
64  {
65      for(int i=0;i<symbolCount;i++)
66      {
67          if(strcmp(table[i].name,name)==0)
68              return table[i].value;
69      }
70
71      fprintf(stderr, "Undefined variable: %s\n", name);
72      return 0;
73  }
74
75  void setSymbol(char *name,double value)
76  {
77      for(int i=0;i<symbolCount;i++)
78      {
79          if(strcmp(table[i].name,name)==0)
80          {
81              table[i].value=value;
82              saveSymbols();
83              return;
84          }
85      }
86
87      strcpy(table[symbolCount].name,name);
88      table[symbolCount].value=value;
89      symbolCount++;
90
91      saveSymbols();
92  }
93
94  %}
95
96  %union
97  {
98      int ival;
99      double dval;
```

```
100     char sval[100];
101 }
102
103 %token <ival> INTEGER
104 %token <dval> DOUBLE
105 %token <sval> ID
106
107 %token SIN COS TAN SQRT LOG
108
109 %type <dval> statement expr term power factor
110
111 %left '+' '-'
112 %left '*' '/'
113 %right '^'
114 %right UMINUS
115
116 %%
117
118 program:
119       lines
120 ;
121
122 lines:
123       lines line
124     |
125 ;
126
127 line:
128       statement '\n'
129     | '\n'
130 ;
131
132 statement:
133
134       ID '=' expr
135       {
136           setSymbol($1,$3);
137           fprintf(output, "%s = %lf\n", $1, $3);
138           $$ = $3;
139       }
140
141     | expr
142       {
143           fprintf(output, "Result = %lf\n", $1);
144           $$ = $1;
145       }
146 ;
147
148 expr:
149
```

```
150       expr '+' term
151       { $$ = $1 + $3; }
152
153     | expr '-' term
154       { $$ = $1 - $3; }
155
156     | term
157       { $$ = $1; }
158 ;
159
160 term:
161
162       term '*' power
163       { $$ = $1 * $3; }
164
165     | term '/' power
166       {
167           if($3 == 0)
168           {
169               fprintf(stderr, "Division by zero\n");
170               $$ = 0;
171           }
172           else
173               $$ = $1 / $3;
174       }
175
176     | power
177       { $$ = $1; }
178 ;
179
180 power:
181
182       factor '^' power
183       { $$ = pow($1, $3); }
184
185     | factor
186       { $$ = $1; }
187 ;
188
189 factor:
190
191       SIN '(' expr ')'
192       { $$ = sin($3); }
193
194     | COS '(' expr ')'
195       { $$ = cos($3); }
196
197     | TAN '(' expr ')'
198       { $$ = tan($3); }
199
```

```
200     | SQRT '(' expr ')'
201       { $$ = sqrt($3); }
202
203     | LOG '(' expr ')'
204       { $$ = log($3); }
205
206     | '-' factor %prec UMINUS
207       { $$ = -$2; }
208
209     | '(' expr ')'
210       { $$ = $2; }
211
212     | INTEGER
213       { $$ = $1; }
214
215     | DOUBLE
216       { $$ = $1; }
217
218     | ID
219       { $$ = getSymbol($1); }
220 ;
221
222 %%
223
224 void yyerror(char *s)
225 {
226     fprintf(stderr, "Syntax Error\n");
227 }
228
229 int main()
230 {
231     loadSymbols();
232
233     yyin = fopen("input.txt", "r");
234     if(!yyin)
235     {
236         fprintf(stderr, "Cannot open input.txt\n");
237         return 1;
238     }
239
240     output = fopen("output.txt", "w");
241     if(!output)
242     {
243         fprintf(stderr, "Cannot open output.txt\n");
244         fclose(yyin);
245         return 1;
246     }
247
248     yyparse();
249
```

```
    fclose(yyin);
    fclose(output);

    return 0;
}
```

4 Input and Output Files

4.1 Input File

The following input file was used to test the scientific calculator parser.

```
2+3*4
PI
E
-5+3
sin(PI/2)
sqrt(25
sqrt(-9)
log(10)
log(0)
x = 10
y = x + 5
y * 2
rad(180)
deg(3.14159265)
tan(PI/2)
marks1=20.5
marks2=10
total=marks1+marks2
x = 10
y = 20.5
z = x + y
power = a ^ b
mixed = 3.14 * 2 ^ 2
```

4.2 Output File

The following output was generated by the parser.

```
Result = 14.000000
Result = 3.141593
Result = 2.718282
Result = -2.000000
Result = 1.000000
```

```
Syntax Error
Invalid sqrt() domain
Result = 2.302585
Invalid log() domain
x = 10.000000
y = 15.000000
Result = 30.000000
Result = 3.141593
Result = 180.000000
Undefined tan()
marks1 = 20.500000
marks2 = 10.000000
total = 30.500000
x = 10.000000
y = 20.500000
z = 30.500000
power = 25.000000
mixed = 12.560000
```

5 Conclusion

Here, the parser can successfully parse the scientific expressions. Here, it can also report various kinds of error. Such as, from the given input and output files we can see the parser can detect the division by zero, syntax error, etc. It can also check the wrong domain, like square root of negetive number, log(0), tan(90) etc values and report error. So , this parser can successfully parse any scientific expressions.